Embla Carousel: Full Step-by-Step Guide

This guide walks through how the product carousel was added to the CareKart Next.js project, from installing Embla to a working row of product cards with a peek of the previous and next cards and hide-at-the-edge arrow buttons. It covers the order you would actually build it in.

No complicated words. Every step is explained in plain language.

What You Will Build

By the end of these steps, the home page will:

1. Show each category's products in a horizontal row that can be dragged or moved with arrow buttons.
2. Show **1 card on phones**, 2 on tablets, and 3 to 6 on larger screens.
3. Show a thin slice ("peek") of the previous and next card, so users can see there is more to scroll.
4. Hide the Prev button on the first position and the Next button on the last position.
5. Keep the products **rendered on the server**. Only the carousel behavior runs in the browser.

Part A: Setup

Step 1: Install Embla

In the Next.js project folder, run:

```
npm i embla-carousel-react
```

What this does:

- Downloads Embla and its React hook (`useEmblaCarousel`).
- Adds it to `package.json`.

Embla is "headless". It only handles the movement (drag, snap, scroll). All the look and feel comes from your own CSS and Bootstrap.

Step 2: Make sure Bootstrap Icons are loaded

The arrow buttons use Bootstrap Icons (`bi bi-chevron-left` and `bi bi-chevron-right`). If they are not already installed:

```
npm i bootstrap-icons
```

Then import the CSS once in `app/layout.tsx`:

```
import "bootstrap-icons/font/bootstrap-icons.css";
```

Do **not** load Bootstrap's own JavaScript carousel for this. It is a different component (one slide at a time). Embla does all the work here.

Step 3: Understand the three layers Embla needs

Embla always needs this structure:

```
viewport   (overflow: hidden, gets the ref)
  └── container   (display: flex)
        └── slide   (does not shrink, has a fixed width)
```

Layer	In this project	Job
Viewport	`<div className="wrap-x overflow-hidden" ref={emblaRef}>`	Clips everything outside the visible area
Container	`<div className="product-grid d-flex flex-nowrap">`	The row that moves
Slide	`<article className="pc">` (the product card)	One product card

Part B: The Carousel Component (Client Side)

Step 4: Create the component

Create `app/components/layout/Carousel.tsx`:

```
"use client";
import { useState, useEffect, useCallback } from "react";
import useEmblaCarousel from "embla-carousel-react";

export default function Carousel({ children }) {
  const [emblaRef, emblaApi] = useEmblaCarousel({
    align: () => 24, // mobile: the active card starts 24px from the left
    containScroll: "trimSnaps",
    slidesToScroll: 1,
    dragFree: false,
    breakpoints: {
      "(min-width: 768px)": { align: () => 40 }, // tablet and up: 40px
    },
  });

  const [canPrev, setCanPrev] = useState(false);
  const [canNext, setCanNext] = useState(false);

  const onSelect = useCallback((api) => {
    setCanPrev(api.canScrollPrev());
    setCanNext(api.canScrollNext());
  }, []);

  useEffect(() => {
    if (!emblaApi) return;
    onSelect(emblaApi);
    emblaApi
      .on("select", onSelect)
      .on("reInit", onSelect)
      .on("settle", onSelect);
    return () => {
      emblaApi
        .off("select", onSelect)
        .off("reInit", onSelect)
        .off("settle", onSelect);
    };
  }, [emblaApi, onSelect]);

  return (
    <div className="position-relative">
      <div className="wrap-x overflow-hidden" ref={emblaRef}>
        <div className="product-grid d-flex flex-nowrap">{children}</div>
      </div>

      <button
        className={`carousel-btn carousel-btn-left ${!canPrev ? "is-hidden" : ""}`}
        onClick={() => emblaApi?.scrollPrev()}
        aria-label="Previous"
      >
        <i className="bi bi-chevron-left"></i>
      </button>
      <button
        className={`carousel-btn carousel-btn-right ${!canNext ? "is-hidden" : ""}`}
        onClick={() => emblaApi?.scrollNext()}
        aria-label="Next"
      >
        <i className="bi bi-chevron-right"></i>
      </button>
    </div>
  );
}
```

Step by step, what happens here:

1. `"use client"` is required because the component uses state, effects and a browser-only hook.
2. `useEmblaCarousel(options)` returns two things:
   - `emblaRef`: attached to the viewport so Embla knows what to control.
   - `emblaApi`: the remote control (`scrollPrev()`, `scrollNext()`, `canScrollPrev()` and so on). It is `undefined` for a moment on first render, so always write `emblaApi?.`.

3. `canPrev` and `canNext` remember whether there is anything left to scroll to in each direction.
4. `onSelect` reads those two answers from Embla and saves them in state.
5. The `useEffect` runs `onSelect` once when Embla is ready, then again on three events:
   - `select`: the active position changed.
   - `reInit`: Embla re-measured after a resize.
   - `settle`: the scrolling animation finished (keeps the buttons correct after a drag).
6. The cleanup function (`off`) removes the listeners when the component disappears, so nothing leaks.
7. The buttons are placed **outside** the viewport. If they were inside, they would be clipped and move with the slides.
8. When `canPrev` or `canNext` is `false`, the class `is-hidden` is added. CSS hides the button (see Part C).

Step 5: What each option means

Option	Value	Meaning
`align`	`() => 24`	Where the active card sits. 24 means "24px from the left edge", which leaves room for the previous card to peek out
`containScroll`	`"trimSnaps"`	No empty space at the start or end. The first and last cards sit flush against the edges
`slidesToScroll`	`1`	Each click moves exactly one card
`dragFree`	`false`	Dragging snaps to a card instead of floating freely
`breakpoints`	`(min-width: 768px)`	Different options for bigger screens (here, a bigger peek)

Why `slidesToScroll: 1` and not `"auto"`: `"auto"` moves a whole screen at once, so the first click jumps far and the next click only moves a little because the end of the row has been reached. One card per click is predictable.

Part C: The CSS (Global Stylesheet)

All of this goes in your global CSS file.

Step 6: Size the cards and reserve space for the peek

```
.product-grid {
  --product-grid-columns: 1;   /* phones: 1 card */
  --product-grid-gap: 0.75rem;
  --product-grid-peek: 24px;   /* must match the JS align value */
  display: flex;
  flex-wrap: nowrap;
  gap: var(--product-grid-gap);
}

.product-grid > .pc {
  flex: 0 0
    calc(
      (
        100% - (var(--product-grid-peek) * 2) -
          ((var(--product-grid-columns) - 1) * var(--product-grid-gap))
      ) / var(--product-grid-columns)
    );
  min-width: 0;
}
```

What this does:

- `--product-grid-columns` is how many **full** cards are visible.
- The card width is: (container width, minus the peek on both sides, minus the gaps) divided by the number of columns.
- `flex: 0 0 ...` means the card never grows or shrinks. Embla needs fixed-size slides.
- `min-width: 0` stops long text (like a product name) from stretching the card.

Step 7: Change the number of cards on bigger screens

```
@media (min-width: 768px) {
  .product-grid {
    --product-grid-columns: 2;
    --product-grid-gap: 1.25rem;
    --product-grid-peek: 40px;
  }
}
@media (min-width: 1024px) {
  .product-grid { --product-grid-columns: 3; }
}
@media (min-width: 1280px) {
  .product-grid { --product-grid-columns: 4; }
}
@media (min-width: 1536px) {
  .product-grid { --product-grid-columns: 5; }
}
@media (min-width: 1920px) {
  .product-grid {
    --product-grid-columns: 6;
    --product-grid-gap: 1.5rem;
  }
}
```

Screen width	Full cards	Peek
under 768px	1	24px
768px and up	2	40px
1024px and up	3	40px
1280px and up	4	40px
1536px and up	5	40px
1920px and up	6	40px

Step 8: Keep CSS and JavaScript peek values in sync

This is the most important rule in the whole setup:

CSS	JavaScript
`--product-grid-peek: 24px` (base)	`align: () => 24`
`--product-grid-peek: 40px` (768px and up)	`breakpoints: { "(min-width: 768px)": { align: () => 40 } }`

If the two numbers differ, the left and right slivers will not match, and the cards will look off-center. If you change one, change the other.

The visible sliver is `peek minus gap`. With a 24px peek and a 12px gap on mobile, you see about 12px of the next card. Increase the peek if you want a bigger sliver.

Step 9: Style the arrow buttons

The base styles (size, circle, white background, shadow, hover color) are already in the stylesheet under `/* ===== Carousel Buttons ===== */`. The parts that matter for behavior are:

```
.carousel-btn {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  transition: opacity 0.2s ease;
}

.carousel-btn-left  { left: 10px; }
.carousel-btn-right { right: 10px; }

.carousel-btn.is-hidden {
  opacity: 0;
  visibility: hidden;    /* removes it from the tab order */
  pointer-events: none;  /* cannot be clicked */
}
```

Why `is-hidden` uses three properties together:

- `opacity: 0` gives a smooth fade.
- `visibility: hidden` stops keyboard users from tabbing to an invisible button.
- `pointer-events: none` stops accidental clicks while it fades.

Step 10: Move the arrows closer to the edge on mobile

To bring the arrows closer to the edge, change the existing values instead of adding new rules:

```
.carousel-btn-left  { left: 2px; }
.carousel-btn-right { right: 2px; }
```

Important: if you paste these as *new* rules **above** the existing `.carousel-btn-left { left: 10px; }` and `.carousel-btn-right { right: 10px; }`, they will not apply. Both rules have the same weight, and when that happens the one written **last** in the file wins. Either edit the original lines directly, or put the new rules **below** them.

To make the new rules win no matter where they are placed, use a stronger selector:

```
.carousel-btn.carousel-btn-left  { left: 2px; }
.carousel-btn.carousel-btn-right { right: 2px; }
```

To change it only on phones, wrap it:

```
@media (max-width: 767px) {
  .carousel-btn.carousel-btn-left  { left: 2px; }
  .carousel-btn.carousel-btn-right { right: 2px; }
}
```

Part D: Using the Carousel on the Page (Server Side)

Step 11: Pass the product cards in as children

In `app/page.tsx` (a server component, with no `"use client"`):

```
import Carousel from "./components/layout/Carousel";

{topCategories.map((category) => (
  <Fragment key={category.id}>
    <div className="sec">
      <h2>{category.name}</h2>
      <a href="index.html">See All</a>
    </div>

    <Carousel>
      {category.products.map((product) => (
        <article key={product.id} className="pc">
          {/* image, name, price, Add to cart form */}
        </article>
      ))}
    </Carousel>
  </Fragment>
))}
```

Why this keeps products server-rendered:

- The page fetches the data on the server (`await fetch(...)`).
- The product cards are created on the server and handed to `Carousel` as `children`.
- `Carousel` is a client component, but it never imports the cards. It only receives them as ready-made content.
- Result: the products are in the first HTML the browser receives (good for SEO and speed). Embla only adds drag and button behavior after the page loads.

Step 12: Rules to keep it that way

- **Never** import the product card into `Carousel.tsx`. Importing it from a `"use client"` file turns it into a client component.
- Do **not** pass the whole `products` array into `Carousel` as a prop. Props to client components get packed into the page data.
- Every card must be a **direct child** of the container with the class `pc`. The CSS rule `.product-grid > .pc` only matches direct children.
- Every card needs a unique `key`.

Step 13: Small fixes to the page code

While adding the carousel, these three things in `page.tsx` are worth fixing:

1. **Use a keyed Fragment instead of `<>`** inside `.map()`. React needs a key on the outer element of each loop item:

```
import { Fragment } from "react";
// ...
<Fragment key={category.id}> ... </Fragment>
```

2. **The "% Off" number is the wrong way round.** `offer_price / base_price * 100` gives the percent you *pay*, not the percent you *save* (a product with prices 705.70 and 708.14 shows "99.66% Off"). Use:

```
{Math.round((1 - product.offer_price / product.base_price) * 100)}% Off
```

3. **Use `className` in JSX**, never `class`. For example, `<i className="bi bi-chevron-left">`.

Part E: Test the Whole Thing

1. Start the Laravel API (`multivendor-ecommerce.test`).
2. Start Next.js: `npm run dev`.
3. Open the home page. Each category should show a row of product cards.
4. **Phone width (about 390px):** one full card with a thin sliver of the next card on the right. The Prev arrow is hidden.
5. Click **Next**. The row moves by exactly one card. A sliver of the previous card now shows on the left, and the Prev arrow appears.
6. Keep clicking until the last card. The Next arrow disappears and the last card sits flush with the right edge.
7. Drag with the mouse or swipe on a touch screen. It should snap to a card.
8. Widen the browser. The number of full cards should grow (2, 3, 4, 5, 6).
9. Resize the window while on the page. The arrows should still show and hide correctly.

Common Problems and Fixes

Problem	Cause	Fix
Nothing happens, console says `useState is not defined`	Hooks not imported	`import { useState, useEffect, useCallback } from "react"`
Buttons never hide	Buttons placed inside the viewport, or events not subscribed	Put buttons outside the `overflow-hidden` div and keep the `useEffect`
First click jumps far, the second moves just a little	`slidesToScroll: "auto"`	Use `slidesToScroll: 1`
Cards are squished or stretched	Slide has no fixed width	Make sure `.product-grid > .pc` has `flex: 0 0 calc(...)` and `min-width: 0`
Cards ignore the new width	Another class (such as a Bootstrap `col-*`) sets its own width	Remove it from the card
Still two cards on mobile	`--product-grid-columns` is still `2` in the base rule	Set the base value to `1`
Slivers uneven on left and right	JS `align` and CSS `--product-grid-peek` differ	Make them the same number
The `left: 2px` / `right: 2px` rule does nothing	An older rule with the same weight is written later in the file	Edit the original lines, or place the new rules below them, or use `.carousel-btn.carousel-btn-left`
Arrow icons do not show	Bootstrap Icons CSS not imported, or `class=` used instead of `className=`	Import the CSS in `layout.tsx` and use `className`
Row looks wrong after images load	Embla measured before images had a size	Give images a fixed height (the `.product-image` rule uses `height: 160px`)
Wrong arrow state after content changes	Embla needs to re-measure	Call `emblaApi.reInit()`

Full Process Overview (Diagram)

```
PART A  Setup
──────────────────────────
Install embla-carousel-react → Load Bootstrap Icons CSS

PART B  Client component
──────────────────────────
Create Carousel.tsx → Add useEmblaCarousel options
→ Track canPrev / canNext → Add Prev / Next buttons (outside viewport)

PART C  CSS
──────────────────────────
Set --product-grid-columns / gap / peek → Size cards with calc()
→ Add breakpoints → Style .carousel-btn and .is-hidden

PART D  Page (server component)
──────────────────────────
Fetch data on the server → Map products into <article className="pc">
→ Wrap them in <Carousel>

HOW IT WORKS AT RUNTIME
──────────────────────────
[Browser requests the home page]
        |
        v
[Server fetches products and renders the cards into HTML]
        |
        v
[Browser shows the cards right away, as a plain flex row]
        |
        v
[Carousel.tsx loads and Embla starts]
        |
        v
[Drag, swipe and arrow buttons now work]
        |
        v
[On every move, Embla fires "select"/"settle" → canPrev/canNext update]
        |
        v
[Arrow gets "is-hidden" when there is nothing left in that direction]
```

Quick Reference Table

Step	File	What It Does
1	`package.json`	Installs `embla-carousel-react`
2	`app/layout.tsx`	Loads Bootstrap Icons CSS for the arrows
4	`app/components/layout/Carousel.tsx`	The client component: Embla setup, state and buttons
5	`Carousel.tsx`	Options: `align`, `containScroll`, `slidesToScroll`, `dragFree`, `breakpoints`
6	Global CSS	`.product-grid` variables and card width formula
7	Global CSS	Media queries for 1 to 6 cards per view
8	CSS and `Carousel.tsx`	Keep `--product-grid-peek` and `align` equal
9	Global CSS	`.carousel-btn` styles and `.is-hidden`
10	Global CSS	Arrow position (`left` / `right`)
11	`app/page.tsx`	Wraps product cards in `<Carousel>` as children

Step	File	What It Does
13	`app/page.tsx`	Keyed Fragment, correct discount %, `className`

Embla API Cheat Sheet

Method	Purpose
`scrollPrev()` / `scrollNext()`	Move one step
`scrollTo(index)`	Jump to a position
`canScrollPrev()` / `canScrollNext()`	Is there anything left in that direction
`selectedScrollSnap()`	Current position number
`scrollSnapList()`	List of all positions
`reInit()`	Re-measure after content or size changes
`on()` / `off()`	Subscribe to or remove events

Events: `init`, `reInit`, `select`, `scroll`, `settle`, `resize`, `slidesInView`, `pointerDown`, `pointerUp`.

Official docs: https://www.embla-carousel.com